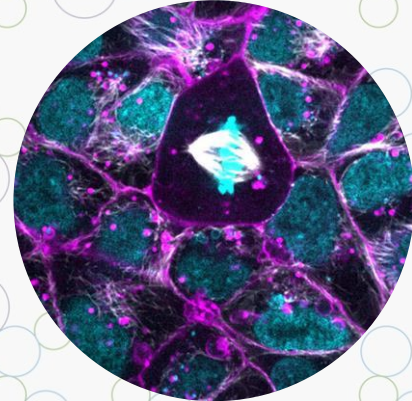




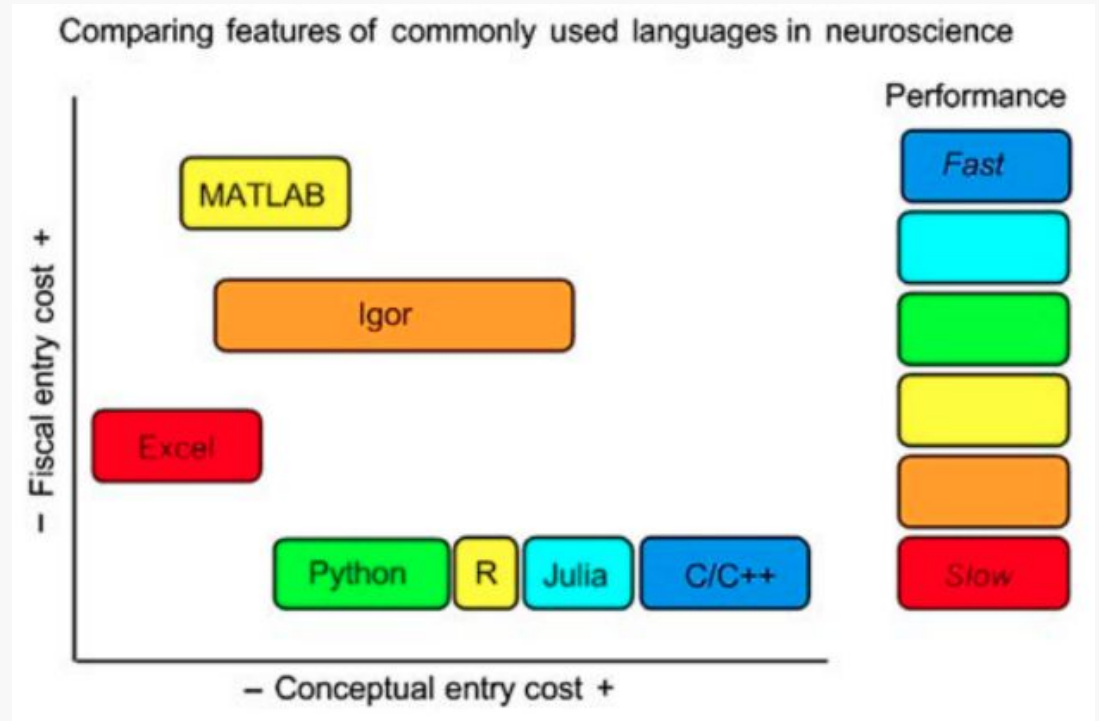
Where Python Lives

Coding with Allen Institute Data: A Python Workshop
for Undergraduate Neuroscience Educators



Considerations for choosing a programming language

- Fiscal & conceptual entry cost (see right)
- Usage in your field
- Ease of use
- *My recommendation:* Python or R



From Wallisch ([2017](#)); See also discussion in Grisham ([2019](#))

Objectives for this session

- Identify various ways of writing and running Python code
 - Learn basics of Python syntax
 - Define three types of variables: integers, floats, and strings
 - Discuss implications of using AI to learn programming
 - Introduce Google Colab platform



There are multiple ways to interact with the Python interpreter

- Command line (terminal)
 - Line-by-line coding
 - Running “Scripts”

“Terminal” comes from the days *before* desktop computers, when a computer occupied a set of cabinets or even an entire room. A terminal was a device with a (text-only) monitor and keyboard whereby a user could control the computer from a distance over a dedicated, wired connection.



A DEC VT100 terminal at the Living Computer Museum (apparently connected to the museum's DEC PDP-11/70 mainframe computer).

Source: [Wikipedia](https://en.wikipedia.org/wiki/DEC_VT100)

Linux



```
Terminal
File Edit View Search Terminal Help
user@linux-computer:~$ python MyPythonScript.py
user@linux-computer:~$ python MyPythonScript.py param1 param2
user@linux-computer:~$
```

Macintosh



```
Users — bash — 86x14
User-Imac:python MyPythonScript.py
User-Imac:python MyPythonScript.py param1 param2
User-Imac:~
```

Windows



```
Command Prompt
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\UserName>python PythonScript.py param1 param2
```



Right-click
Edit with IDLE

```
PythonScript.py - C:\Users\Tim Stevens\Desktop\PythonScript.py
File Edit Format Run Options Windows Help

def calcSeqIdentity(seqA, seqB):
    numPlaces = min(len(seqA), len(seqB))
    score = 0.0
    for i in range(numPlaces):
        if seqA[i] == seqB[i]:
            score += 1.0
    return 100.0 * score/numPlaces

Ln: 1 Col: 0
```

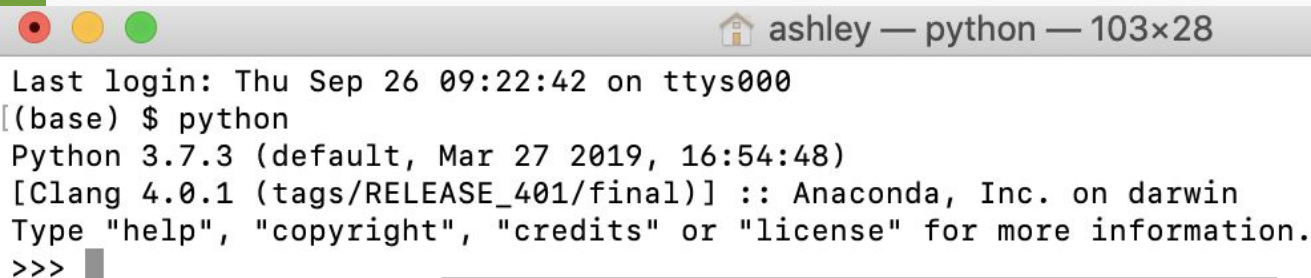
Running a Python script from
different operating systems

(from <http://www.cambridge.org/pythonforbiology>)

If you have a Mac

- Macs ship with Python already installed.
- You can check which version by opening **Terminal** & typing
`python --version`

Windows does not require Python natively and so it is not typically pre-installed.

A screenshot of a macOS Terminal window. The title bar shows three colored window control buttons (red, yellow, green) on the left, a home icon, and the text 'ashley — python — 103x28'. The terminal content shows the command 'python' being executed, resulting in the output: 'Python 3.7.3 (default, Mar 27 2019, 16:54:48) [Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin'. It then prompts the user to type 'help', 'copyright', 'credits', or 'license' for more information. The prompt '>>>' is followed by a small grey square cursor.

```
Last login: Thu Sep 26 09:22:42 on ttys000
(base) $ python
Python 3.7.3 (default, Mar 27 2019, 16:54:48)
[Clang 4.0.1 (tags/RELEASE_401/final)] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

The “>>>” tells you you’re inside the Python prompt, and the computer is ready for some code!

There are multiple ways to interact with the Python interpreter

- Command line
 - Line-by-line coding
 - Running “Scripts”
- Integrated Development Environments
 - Folks have strong opinions about these, and each have pros/cons.
 - A few good options are:
 - Visual Code (<https://code.visualstudio.com/download>)
 - Spyder (Included with Anaconda)
- Jupyter Notebook (e.g., via Google Colab) — *most of what we'll do in this course*

More on these on
Day 4!

Objectives for this session

- Identify various ways of writing and running Python code
- **Learn basics of Python syntax**
- **Define three types of variables: integers, floats, and strings**
- Discuss implications of using AI to learn programming
- Introduce Google Colab platform

There are different types of programming languages, each with their own syntax, or rules.

- **Syntax:** the rules of a programming language
 - Includes punctuation, spacing, indentation, etc.
- Each language has strengths & weaknesses.
- Regardless, each language ultimately needs to communicate with the hardware of the computer, in 1's and 0's.
 - It's similar to DNA! And similar to DNA, we don't often describe it in individual base pairs. Instead we describe genes and describe DNA in a higher level way.

Storing values

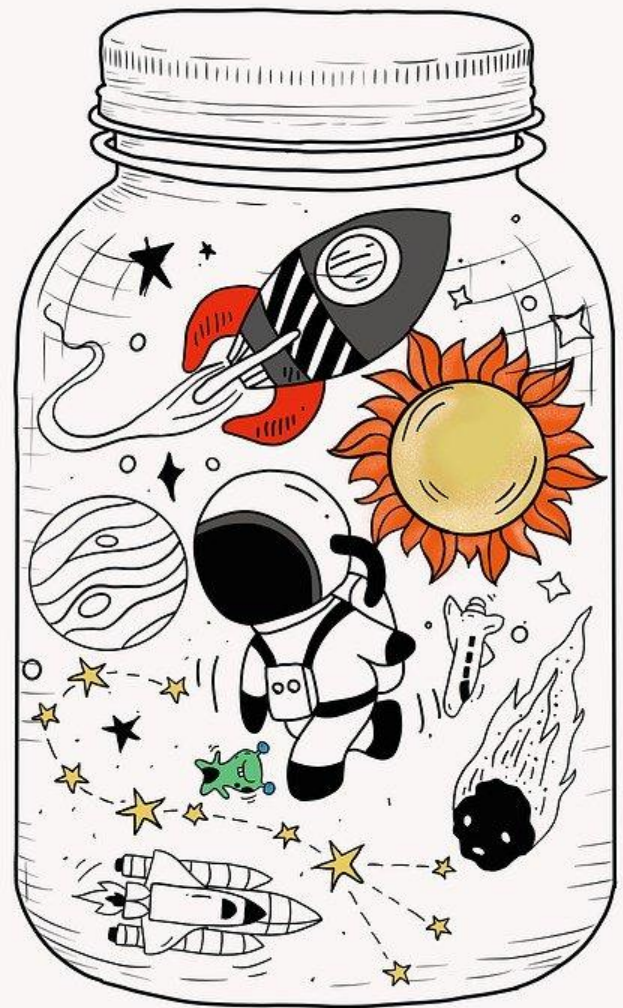
We can store values in variables, e.g.:

```
variable_1 = 48
```

 ↑ ↑
 name value

Variables can be text, integers, or floats (with decimals), e.g.:

```
text_string = 'hello'
```



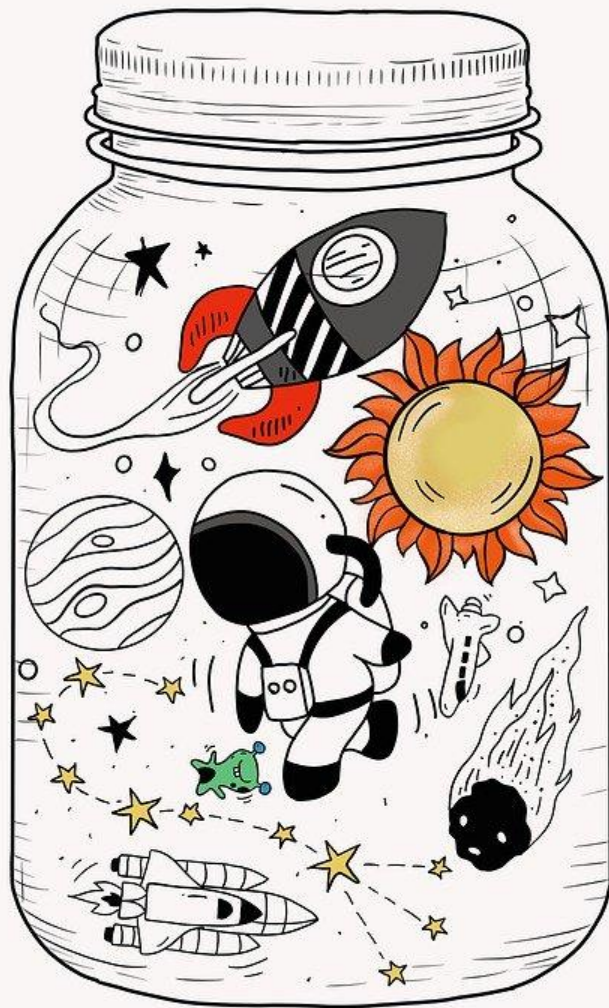
Storing values

We can store values in variables, e.g.:

```
variable_1 = 48
```

We use an equal sign to *assign* the value to a name, but it's not the same thing as saying they are equal.

In other words, we're storing that value in the variable. (Think of them like cookie jars)



Creating new variables

- Names are always on the left of the '=', values are always on the right
- Pick names that describe the data / value that they store
- Make variable names as **descriptive** and **concise** as possible (this is an art!)
- Variables cannot be Python keywords:

```
[>>> import keyword  
[>>> print(keyword.kwlist)  
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def',  
 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda',  
 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']  
>>> █
```

(There are other rules for variable names....)

**Python has many variable types,
and each function a little bit
differently.**

**Understanding your variable type
is crucial for working with it.**



Built-in simple variable types in Python

Type	Example	Description
<code>int</code>	<code>x = 1</code>	integers (i.e., whole numbers)
<code>float</code>	<code>x = 1.0</code>	floating-point numbers (i.e., real numbers)
<code>complex</code>	<code>x = 1 + 2j</code>	Complex numbers (i.e., numbers with real and imaginary part)
<code>bool</code>	<code>x = True</code>	Boolean: True/False values
<code>str</code>	<code>x = 'abc'</code>	String: characters or text
<code>NoneType</code>	<code>x = None</code>	Special object indicating nulls

Integers, strings, floats

function to convert to integer

- **Integers** (**int**): any whole number
- **Float** (**float**): any number with a decimal point (floating point number)
- **String** (**str**): letters, numbers, symbols, spaces
 - Represented by matching beginning & ending quotes
 - Quotes can be single or double; use single *within* double
 - Use \ to ignore single quote
 - Concatenate strings with +

Checking variable types

This is a very useful troubleshooting step!

- You can check what type your variable (**a**) is by using **type(a)**
 - Alternatively, we can use:

```
>>> type(a) is float
```


or

```
>>> isinstance(x, float)
```
- Python lets you change the type of variables, however, ***you cannot combine types.***
- Use **del** to delete variables

It's important to know the precision of your variables.

In most datasets, we are working with floats.



Autopsy Report:

Dr. Andrew Esty

Time of Death: 03/16 11:53

Cause of Death: Rounding Errors

Use
`print()`
often!



Mathieu Alain @mathieualain@mastodon.social
@miniapleur



People: What debugging tool you use ?

me: `print()`



[Original tweet](#)



Yani Bellini Saibene

@yabellini@fosstodon.org

7m*

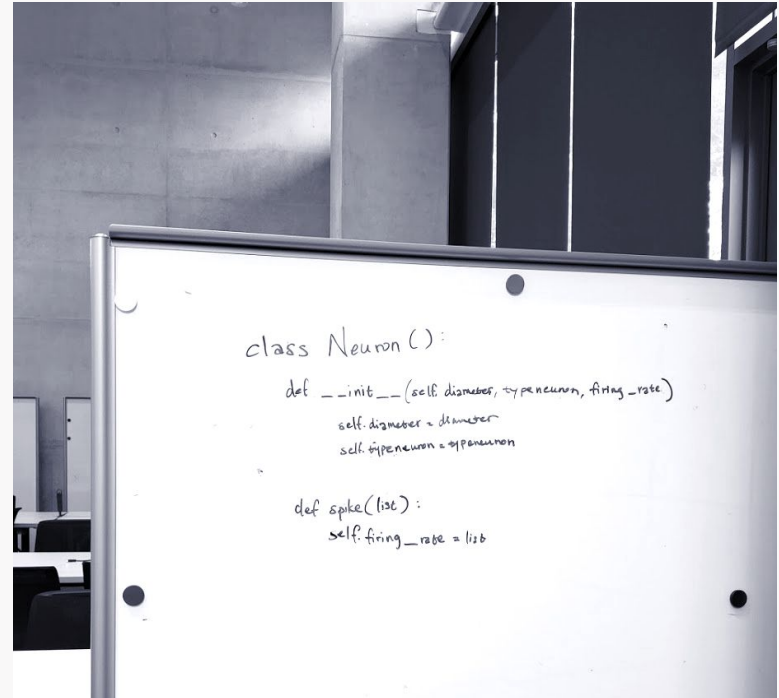
@grimalkina my @thecarpentries rubber duck and my Groot. They are very helpful 😊

I also love to be the rubber duck for my team mates or friends 🦹

[Translate](#)



Rubber ducking & pseudocoding!!!



Objectives for this session

- Identify various ways of writing and running Python code
- Learn basics of Python syntax
- Define three types of variables: integers, floats, and strings
- **Discuss implications of using AI to learn programming**
- **Introduce Google Colab platform**

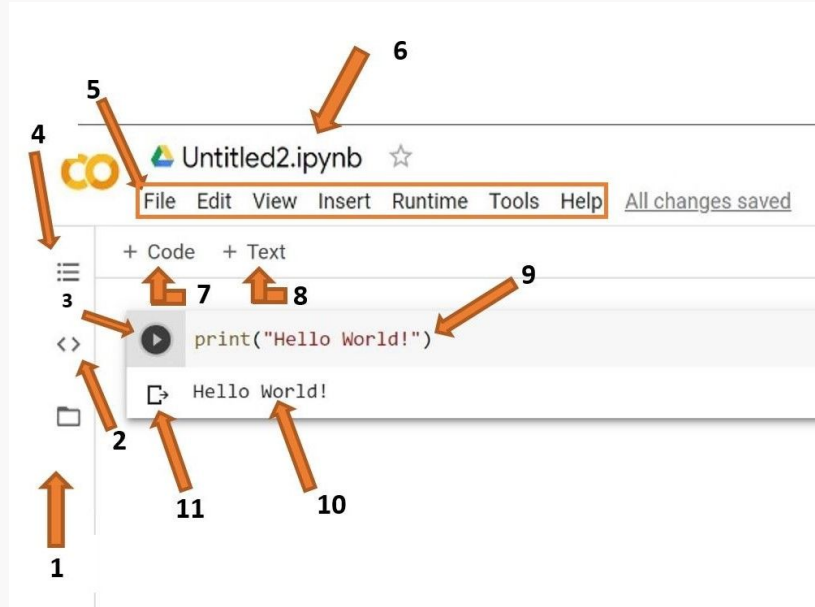
Using Google Colab

Options for launching Colab

- Go to <https://colab.research.google.com/>
- Launch a file directly from Google Drive
- Use an “Open in Colab” button (i.e. from GitHub)
 - GitHub is *just where we are hosting our files* (and it also provides a useful preview)



Quick Colab Orientation



Modified from

https://runestone.academy/ns/books/published/htmlads/PythonReview/google_colab.html

1. **Files:** Upload datasets and other files from both your computer and Google Drive
2. **Code Snippets:** Prewritten snippets of code for different functionalities like adding new libraries or referencing one cell from another.
3. **Run Cell:** This is the run button. Clicking this will run any code that is inserted in the cell beside it. You can use the shortcut shift+enter to run the current cell and exit to a new one.
4. **Table of Contents:** Create and traverse different sections inside of your notebook. Sections allow you to organize your code and improve readability.
5. **Menu Bar:** Manipulate the entire file or add new files.
6. **File Name:** This is the name of your file. You can click on it to change the name. Do not edit the extension (.ipynb) while editing the file name as this might make your file unopenable.
7. **Insert Code Cell:** Add a code cell below the cell you currently have selected.
8. **Insert Text Cell:** Add a text cell below the cell you currently have selected.
9. **Cell:** This is the cell. This is where you can write your code or add text depending on the type of cell it is.
10. **Output:** This is the output of your code, including any errors, will be shown.
11. **Clear Output:** This button will remove the output.

Quick Colab Orientation



Modified from

https://runestone.academy/ns/books/published/htmlads/PythonReview/google_colab.html

12. **Ram and Disk:** All of the code you write will run on Google's computer, and you will only see the output. This means that even if you have a slow computer, running big chunks of code will not be an issue. Google only allots a certain amount of Ram and Disk space for each *user*, so be mindful of that as you work on larger projects.
13. **Link to Cell:** This button will create a URL that will link to the cell you have selected.
14. **Comment:** This button will allow you to create a comment on the selected cell. Note that this will be a comment on (about) the cell and not a comment in the cell.
15. **Settings:** This button will allow you to change the Theme of the notebook, font type, and size, indentation width, etc.
16. **Delete Cell:** This button will delete the selected cell.
17. **More Options:** Contains options to cut and copy a cell as well as the option to add form and hide code.

Some preliminary thoughts on the use of AI assistants

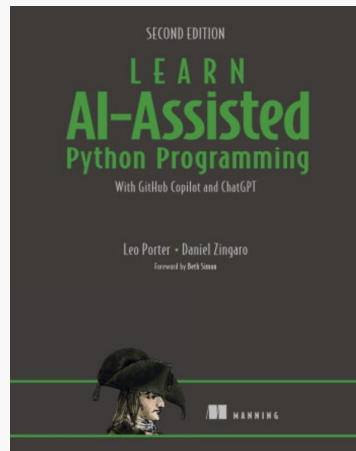
Oh yeah, what about AI assistants?

You (and your students) can use it to learn!

(...how you integrate it into your class depends on your learning objectives!)

Learn AI-Assisted Python Programming
(Second Edition)

by Leo Porter and Daniel Zingaro



AI assistants can do for us:

- Explain code — *“What is this line of code doing?”*
 - Note that this can be wrong...
- Make code easier — *“Simplify this program so that it is easier to read.”*
- Fixing bugs — *“Why isn’t this program _____?”*



Inspired by Porter & Zingaro,
[Learn AI-Assisted Python Programming](#)

Some terms

An **AI-assistant** is an artificial intelligence agent (a computer program) that responds to normal human inputs like speech and text with human-like answers (e.g., Alexa, Siri)

There are several code writing AI assistants, such as Google Copilot.

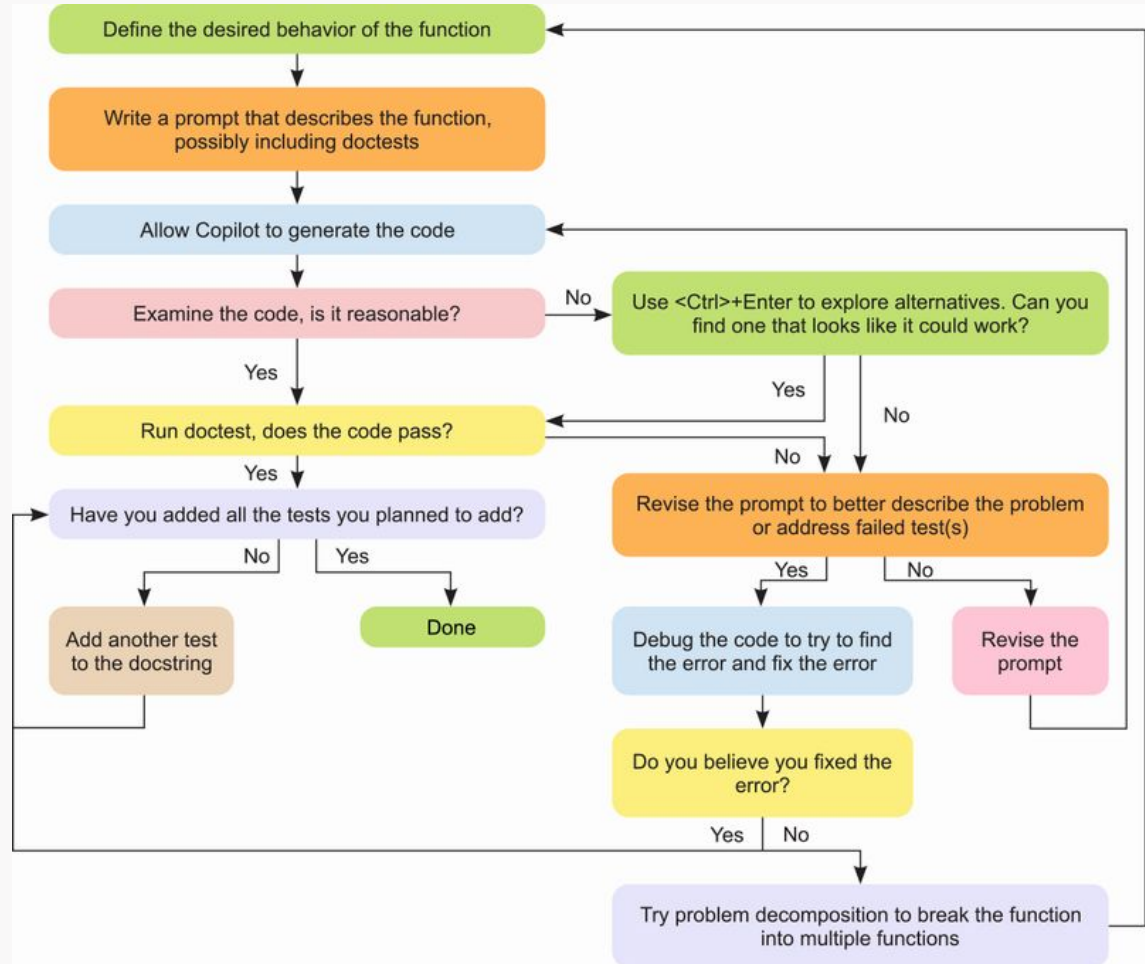
AI assistants are driven by **large language models (LLMs)**, which store information about relationships between words, including which words make sense in certain contexts, and use this to predict the best sequence of words to respond to a prompt.

The **prompt** is what you tell the AI assistant, describing the program you want.

Prompt engineering is the skill of writing tactful and precise prompts.

From Porter & Zingaro,
[Learn AI-Assisted Python Programming](#)

Coding workflow with an AI assistant

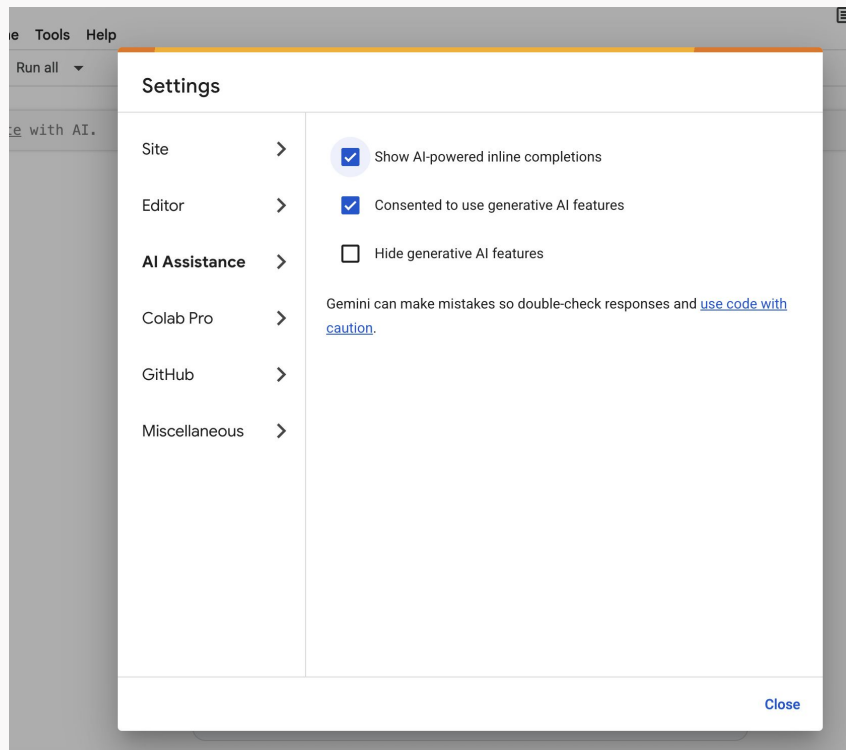


Using AI in Colab

We can choose to use AI-powered inline completions or not.

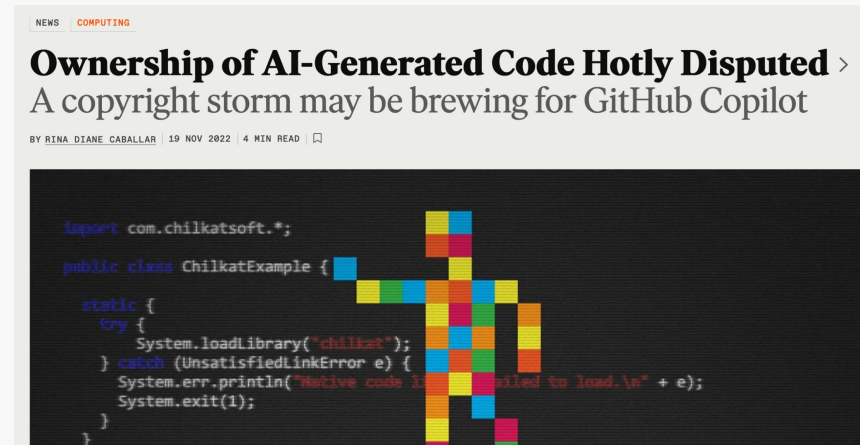
For the purposes of this workshop, we'll turn them off!

Tools > Settings > AI Assistance



Issues with generative AI

- It doesn't know what it doesn't know → this leads to **fabrication**
- It is **biased** because it replicates biases in its training set (try “generate a list of names”)
- Ongoing copyright issues — LLMs are trained on other people's code!
- Societal concerns about jobs, the human condition, etc.
- Environmental concerns



<https://spectrum.ieee.org/ai-code-generation-ownership>

Small group check in:

How do *you* feel about using AI for yourself (for learning programming)?

How do you feel about your students using it?

Let's launch Colab and dig in!

Resources

Jupyter Notebooks:

- [Official Jupyter documentation](#)
- [Example notebooks](#)
- [A Gallery of Interesting Jupyter Notebooks](#)
- [Software Carpentry: Running & Quitting Jupyter Notebooks](#)

Resources

General Python

- [A List of Good Python YouTube Channels](#)
- [CodeAcademy Python Syntax Cheatsheet](#)
- [Python Fundamentals](#) (Software Carpentries)
- [Error types in Python](#)

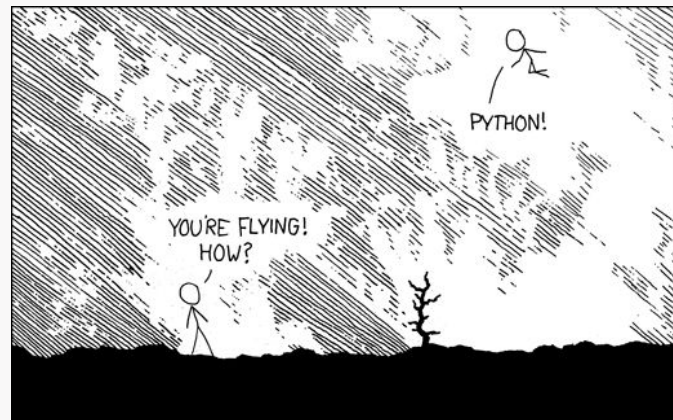
On Linux commands

- [60 essential Linux commands](#)
- [IPython and Shell Commands | Python Data Science Handbook](#)

Let's see if Python can make us fly...

Optional activity: if you have Python installed on your computer, open a terminal and type

```
$ python
>>> import antigravity
```



<https://xkcd.com/353/>

Code: <https://github.com/python/cpython/blob/main/Lib/antigravity.py#L7-L17>

Explanation: <https://martinapugliese.github.io/tech/python-antigravity/>

Useful Linux Commands

In Jupyter Notebook, add a **!** in front to use these.
E.g., **!pwd**

Command	Description
<code>pwd</code>	Print working directory
<code>ls</code>	List contents
<code>cd</code>	Change directory
<code>cp</code>	Copy files from the current directory to a different directory
<code>mv</code>	Move or rename files
<code>mkdir</code>	Make a directory
<code>touch</code>	Create a blank file

Today's agenda

Time (PT)	Description
9:00am	Arrival
9:00am – 10:00am	Breakfast & Introductions
10:00am – 11:00am	Where Does Python Live? <i>Ashley Juavinett</i>
11:00am – 12:30pm	Basic Python I: Objects and Data Structures Rachel Penton Theresa McKim
12:30pm – 1:30pm	Lunch
1:30pm – 2:15pm	Building Tour of the Allen Institute <i>Kaitlyn Casimo</i>
2:15pm – 3:15pm	Lab Tours & Pedagogy Unpacking Time <i>Gathering Space & Lab 420</i> <i>Note: Wear closed toe shoes & long pants to enter lab.</i>
3:15pm – 4:30pm	Basic Python II: Control Flow and Functions <i>Ashley Juavinett</i>
4:30pm – 5:00pm	End of Day Reflections